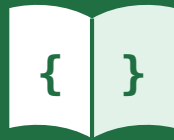

Bitirme Projesi — Kitapçı Platformu

Ödevler

Spring Boot Edu



2026-09-25

İçindekiler

Nasıl Çalışılır	2
Ödev 1 — Siparişler için Idempotency Anahtarı (Orta)	2
Ödev 2 — Siparişi İptal Etmek (Orta)	3
Ödev 3 — Katalog Yeniden Başlarken Ayakta Kalmak (Kolay)	3
Ek Görev (İsteğe Bağlı)	4

Nasıl Çalışılır

1. Başlangıç kodu capstone/exercise/order-service/ içindedir: veritabanı migration'ı, entity alanları ve endpoint'leri hazırlanmış bir sipariş servisi kopyası. TODO yorumlarını bulun.
2. Testler PostgreSQL ve Kafka'yı Testcontainers ile, sahte bir kataloğu ise in-process başlatır (Docker gerekir, başka servis gerekmez):

```
./mvnw -Pexercises -pl capstone/exercise/order-service -am test
```

3. Tüm testler yeşil olduğunda ödev tamamdır — sipariş servisinin kendi testleri de yeşil kalmalıdır.
4. Takılırsanız önce ipuçlarını okuyun, sonra capstone/solution/order-service/ içindeki çözüme bakın.

Üç ödevin de sorusu aynıdır: bir istek, bir mesaj ya da bir servis **tekrarlandığında veya hiç gelmediğinde** ne olur?

Ödev 1 — Siparişler için Idempotency Anahtarı (Orta)

Hedef: POST /api/orders isteği zaman aşımına uğrayan bir istemci isteği tekrar gönderir. İki kez sipariş vermemelidir. İstemci bir Idempotency-Key header'ı gönderir; aynı müşterinin aynı anahtarı ilk siparişi geri verir.

Görevler:

- TODO Exercise 1 (order.OrderRepository) — müşteri ID'si ve idempotency anahtarıyla bir sipariş bulan türetilmiş (derived) sorgu.
- TODO Exercise 1 (order.OrderService.findByKey) — anahtar yoksa önceki sipariş de yoktur; anahtar varsa onu arayın.

Hazır olanlar: V3__status_and_idempotency.sql migration'ı (sütun ve (customer_id, idempotency_key) üzerinde unique kısmi indeks), Order'daki alan ve controller: bulunan bir sipariş 201 yerine 200 ile cevaplanır.

İpuçları:

- Anahtar bir müşteriye aittir: iki müşteri aynı anahtarı kullanabilir.
- place() neden DataIntegrityViolationException'ı da yakalıyor? Aynı milisaniyede aynı anahtarla gelen iki isteği düşünün — unique indeks yalnızca birinin kazanmasına izin verir.

Kabul kriterleri: Exercise1IdempotencyTest'teki 3 testin hepsi geçer.

Tahmini süre: 30 dakika

Ödev 2 — Siparişi İptal Etmek (Orta)

Hedef: DELETE /api/orders/{id}, giriş yapmış müşterinin bir siparişini iptal eder. Katalog stoğu geri alır, sipariş CANCELLED olarak işaretlenir ve outbox üzerinden bir OrderCancelled event'i yayınlanır.

Görevler (order.OrderService.cancel):

- TODO Exercise 2 — bilinmeyen bir sipariş ya da başkasının siparişi → OrderNotFoundException (404); zaten iptal edilmiş bir sipariş → OrderAlreadyCancelledException (409).
- Stoğu stock.release(orderId) ile geri verin.
- Tek bir transaction'da: order.cancel() ve outbox'a bir OrderCancelled event'i (OrderCancelled.TOPIC, anahtar = sipariş ID'si).

İpuçları:

- Önce mi geri vermeli, önce mi kaydetmeli? ReleaseStock katalogda idempotent'tir: iki kez geri vermek hiçbir şeyi değiştirmez. İki adımın hangi sırası, ikinci adımda bir hata olduktan sonra tekrarlanan bir DELETE'i güvenli yapar?
- OrderCancelled'ın kendi topic'i var. Javadoc'unu okuyun: neden bookstore.orders değil? (Arama servisinin o topic'i nasıl okuduğuna bakın.)
- Siparişi transaction içinde yeniden yükleyin; yönetilen (managed) bir entity'nin değişiklikleri commit'te kaydedilir.

Kabul kriterleri: Exercise2CancellationTest'teki 4 testin hepsi geçer.

Tahmini süre: 45 dakika

Ödev 3 — Katalog Yeniden Başlarken Ayakta Kalmak (Kolay)

Hedef: katalog yeniden başlarken sipariş servisi bir an için UNAVAILABLE alır. Müşteri 503 almadan önce rezervasyon iki kez yeniden denenir. "Yeterli stok yok" asla yeniden denenmez.

Görevler:

- TODO Exercise 3 (stock.ResilienceConfiguration) — Spring'in resilience annotation'larını açın.
- TODO Exercise 3 (stock.StockClient.reserve) — yalnızca CatalogUnavailableException'da yeniden deneyin: 1 çağrı + 2 tekrar, kısa ve artan bir bekleme ile.

İpuçları:

- Modül 04, ders 3.6: @EnableResilientMethods ve @Retryable (Spring Framework 7, ek kütüphane yok).

- reserve'ü yeniden denemek neden güvenli, ama anahtarsız bir ödemeyi yeniden denemek neden güvenli olmazdı? (Katalog rezervasyonu sipariş ID'si altında saklar.)
- Test, kapalı kalan bir katalog için tam olarak 3 deneme bekler.

Kabul kriterleri: Exercise3RetryTest'teki 3 testin hepsi geçer.

Tahmini süre: 20 dakika

Ek Görev (İsteğe Bağlı)

1. **Outbox üzerinden trace.** OrderPlaced yeni bir trace başlatır (rehber bölüm 3.8). O anki span'in W3C traceparent değerini outbox'ta yeni bir sütunda saklayın ve relay kaydı gönderirken bu trace'i sürdürsün. Grafana'da sipariş, gateway'den bildirim listener'ına kadar tek bir trace olmalı.
2. **Arama indeksinde iptaller.** Arama servisi bookstore.order-cancellations'ı tüket-sin ve miktarları sold'dan düşsün — recordSale gibi idempotent olarak. Önce Order-Cancelled'ı contracts'a taşıyın.